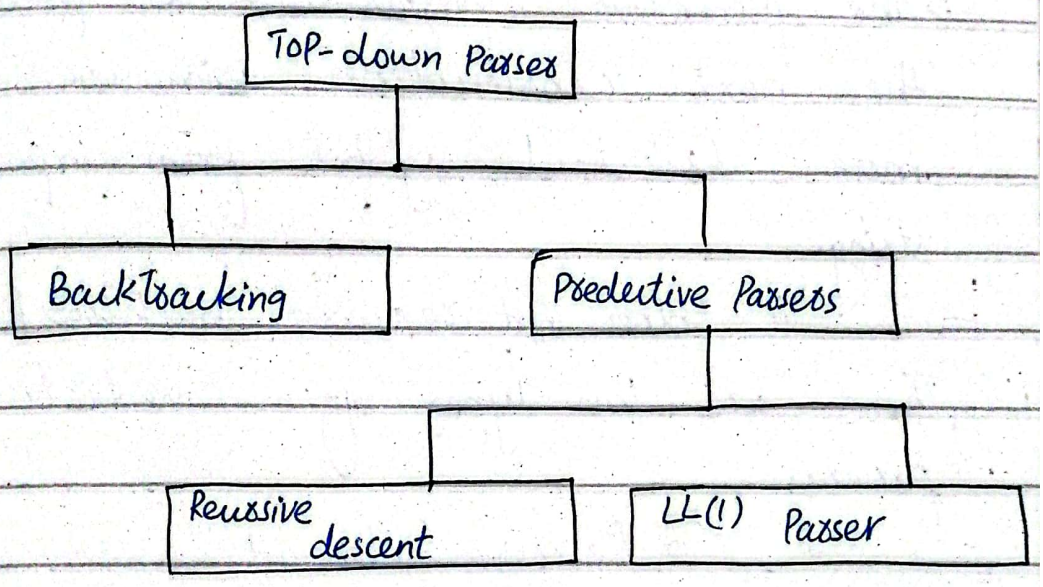
Q. Discuss the types of Top-down passer? And Elaborate each one?

There are two types of top-down passes by which passing can be done.



1) Backtracking passes:

A backtracking passes will try different production rules to find the match for the input string by backtracking each time. The backtracking is powerful than predictive passing. But this technique is slower than predictive passing and it requires exponential time in general. Hence,

backtracking is not preferred for practical compilers.

2) Predictive Passers

⇒ The predictive passers tries to predict the next construction using one or more lookahead symbols from input string.

⇒ It does not require backtracking.

These are two types of predictive passers:

- Recursive descent
- LL(1) Passers

Recursive Descent Passers

⇒ A passers that uses collection of recursive procedures for passing the given input string is called Recursive descent passers (RDP).

⇒ In this type of passers the CFG is used to build the recursive routines.

⇒ Where each routine implements one of the non-terminals of the grammar.

⇒ Thus the structure of the resulting program closely mirrors that of the grammer it recognizes.

⇒ The R.H.S of the production rule is directly converted to the program.

⇒ For each non-terminal a separate procedure is written and body of the procedure (code) is R.H.S of the corresponding non-terminal.

Basic Steps for the Construction of RD Passers :-

⇒ The R.H.S of the rule is directly converted into program code symbol by symbol.

⇒ if the input symbol is non-terminal then a call to the procedure corresponding to the non-terminal is made.

⇒ if the input symbol is terminal then it is matched with the lookahead from input. The lookahead pointer has to be advanced on matching of the input symbol.

⇒ if the input rule has many alternates then all these alternates has to be combined into a single body of procedure.

⇒ The parses should be activated by a procedure corresponding to the start symbol.

Example:

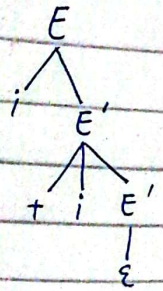
Construct recursive descent passes for grammer.

$E \rightarrow iE'$

$E' \rightarrow +iE' / \varepsilon$

```
E()
{
    if (look_ahead == 'i')
    {
        match('i');
        E'();
    }
```

```
E'()
{
    if (look_ahead == '+')
    {
        match('+');
        match('i');
        E'();
    }
    else
        return;
}
match(char c)
{
    if (look_ahead == c)
        look_ahead = getchar();
    else
        printf("Error!");
}
main()
{
    E();
    if (look_ahead == '$')
        printf("Parsing Successful");
}
```



Q. Construct descent parser

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid id$

```
E()
{
    T();
    Eprime();
}
Eprime()
{
    if(look_ahead == '+')
    {
        match('+');
        T();
        Eprime();
    }
    else
        return;
}
T()
{
    F()
    Eprime();
}
Eprime()
{
    if(look_ahead == '*')
    {
        match('*');
        F();
        Eprime();
    }
    else
        return;
}
F()
{
    if(look_ahead = '(')
    {
        match('(');
        E();
        if(look_ahead == ')')
            match(')');
    }
```

```
else if (look_ahead == 'id')
        match('id');
}

match (char c)
{
    if (look_ahead == c)
        look_ahead = getchar();
    else
        printf("error");
}

main()
{
    E();
    if (look_ahead == '$')
        printf("Passing successful");
}
```

Q. Consider the following grammer

$E \rightarrow T+E \mid T$

$T \rightarrow V*T \mid V$

$V \rightarrow id$

Write down procedures for non-terminals of the grammer to make a recursive descent passer?

```
Procedure E()
{
    T();
    if (look_ahead == '+')
    {
        match('+');
        E();
    }
    else
        error();
}
```

```
Procedure T()
{
    V();
    if (look_ahead == '*')
    {
        match('*');
        T();
    }
    else
        error();
}

Procedure V()
{
    if (look_ahead == 'id')
    {
        match('id');
    }
    else
        error();
}
```

```
Procedure match (token t)
{
    if (look_ahead == t)
        look_ahead = next_token;
    else
        error;
}

Procedure error()
{
    Printf ("Error!");
}

main ()
{
    E();
    if (look_ahead == '$')
        printf ("Parsing Successful");
}
```

Q. Write down C program for recursive descent parser for:

$S \rightarrow ABC$
$A \rightarrow 0A \mid 1 \mid \wedge$
$B \rightarrow 1B \mid \wedge$
$C \rightarrow 1C0 \mid \wedge$

```
procedure S()
{
    A();
    B();
    C();
}
procedure A()
{
    if(look-ahead == '0')
    {
        match('0');
        A();
        if(look_ahead == '1')
            match('1');
    }
    else
        return;
}
procedure B()
{
    if( look_ahead == '1')
    {
        match('1');
        B();
    }
else
    return;
}
procedure C()
{
    if( look_ahead == '1')
    {
        match('1');
        C();
        if( look_ahead == '0')
            match('0');
    }
else
    return;
}
```

```
procedure match(token t)
{
  if(look_ahead == t)
    look_ahead = next_token;
  else
    error;
}

main()
{
  S();
  if(look_ahead == '$')
    printf("Passing Successful");
}
```

Q. Implement the following grammer using recursive descent passer.

$S \rightarrow Aa \mid bAc \mid bBa$

$A \rightarrow d$

$B \rightarrow d$

```
procedure S()
{
  if(lookahead == 'b')
  {
    B(); // or A() because both gives 'd'
    if(lookahead == 'c')
      match('c');
    else if(lookahead == 'a')
      match('a');
    else if(lookahead == '$')
      declare SUCCESS;
  }
  else
  {
    A();
    if(lookahead == 'a')
      match('a');
  }
}
```

```
Procedure A()
{
if (lookahead == 'd')
{
    match('d');
}
else
    error ();
}
Procedure B()
{
if (lookahead == 'd')
    match('d');
else
    error();
}
```

Q. Construct recursive descent passer for following grammer.

$E \rightarrow TA$
$A \rightarrow +TA$
$A \rightarrow \varepsilon$
$T \rightarrow FB$
$B \rightarrow *FB$
$B \rightarrow \varepsilon$
$F \rightarrow (E)$
$F \rightarrow id$

Solution:

The routines for recursive passes

```
E()
{
  T();
  A();
}
A()
{
   if( look_ahead == '+')
   {
        match('+');
        T();
        A();
   }
```

```
}
T()
{
  F();
  B();
}
B()
{
  if (look_ahead == '*')
  {
    match ('*');
    F();
    B();
  }
  else
    return;
}
F()
{
  if (lookahead == '(')
  {
    match ('(');
    E();
    match (')');
  }
  else
    match ('id');
}
match (char t)
{
  if (look_ahead == t)
    look_ahead = getchar();
  else
    printf (" Error");
}
main ()
{
  E();
  if (look_ahead == '$')
    printf (" Passing Successful");
}
```

Advantages of recursive descent passer:

⇒ Recursive descent passers are simple to build.
⇒ Recursive descent passers can be constructed with the help of passe tree.

Limitations of recursive descent passes:

⇒ Not very efficient.
⇒ These are chances that the program for RD passes may enter in an infinite loop for some input.
⇒ RD passes can not provide good error messaging.
⇒ it is difficult to passe the string if lookahead symbol is arbitrarily long.

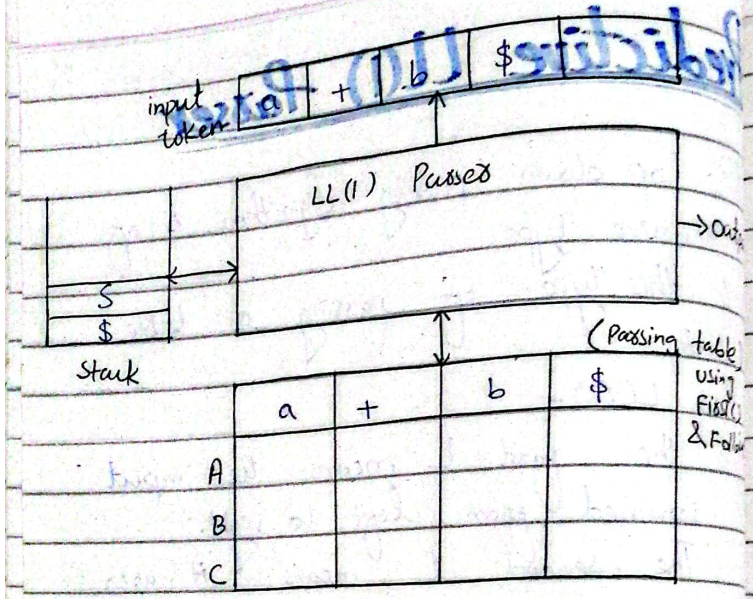
Predictive LL(1) Passer

⇒ This top-down passing algorithm is of non-recursive type.
⇒ In this type of passing a table is built.
⇒ For LL(1)-
- the first L means the input is scanned from left to right.
- The second L means it uses leftmost derivation for input string.
- And the number 1 in the input symbol means it uses only one input symbol (lookahead) to predict the passing process.

Simple block diagram or Model for LL(1) Passer :-

⇒ The data structures used by LL(1) are:

(1) Input buffer (2) Stack
(3) Passing table



⇒ The LL(1) passer uses input buffer to store the input tokens.

⇒ The stack is used to hold the left sentential form.

⇒ The symbols in R.H.S of rule are pushed into the stack in reverse order. i.e. from right to left.

⇒ Thus use of stack makes the algorithm non-recursive.

⇒ The table is basically a two dimensional array.

⇒ The table has row for non-terminal and column for terminals.

⇒ The table can be represented as M[A, a] where A is a non-terminal and a is current input symbol.

Working:-

- The passing program read top of the stack and a current input symbol. With the help of these two symbols the passing action is determined. The passing actions can be:

Top of stack	input token	Passing action
$	$	Passing Successful halt.
a	A	Pop a and advance lookahead to next token.
a	B	Error.
A	a	Refer table M[A,a] if entry at [M,a] is error report Error.

A	a	Refer table M[A, a] if entry M[A, a] is A → PQR then POP A then Push R, then Push Q then push P.

- The passer consults the table M[A, a] each time while taking the passing actions hence this type of passing method is called table driven passing algorithm.
- The configuration of LL(1) passer can be defined by top of the stack and a lookahead token.
- One by one configuration is performed and the input is successfully passed if the passer reaches the halting configuration.
- When the stack is empty and next token is $ then it corresponds to successful passe.

Construction of predictive LL(1) Passer:

=> The construction of predictive LL(1) passer is based on two very important functions and those are:

- FIRST( )
- FOLLOW( )

=> For construction of predictive LL(1) passer the following steps are followed:-

1) Computation of FIRST and FOLLOW function.
2) Construct the Predictive parsing table using FIRST and FOLLOW functions.
3) Passe the input string with the help of predictive Passing Table.

First function:-

=> FIRST(α) is a set of terminal symbols that are first symbols appearing at R.H.S in derivation of α.

in FIRST(α).

⇒ Following are the rules used to compute the FIRST functions.

- if the terminal symbol a the FIRST(a)={a}
- if there is a rule X→ε then FIRST(X)={ε}
- For the rule A→ $X_1X_2X_3 \dots X_K$

$FIRST(A) = (FIRST(X_1) \cup FIRST(X_2) \cup \dots \cup FIRST(X_K))$.

Where k $X_j \le n$ such that $1 \le j \le k-1$.

Follow Function:-

⇒ FOLLOW(A) is defined as the set of terminal symbols that appear immediately to the right of A. In follow ε will never include. - focus just on right side.

In other words:

⇒ FOLLOW(A) = $\{a \mid S \overset{*}{\Rightarrow} \alpha A a \beta$ where α and β are some grammer symbols may be terminal or non-terminal.

⇒ The rules of Computing FOLLOW function are given as:

1. For the start symbol S place $ in FOLLOW(S).

2. If there is a production A→αBβ then everything in FIRST(B) without ε is to be placed in FOLLOW(B).

3. if there is a production A→αB or A→αB and FIRST(B) = {ε} then FOLLOW(A) = FOLLOW(B) or Follow(B) = FOLLOW(A). That means everything in FOLLOW(A) is in FOLLOW(B).

Q. **Find the first and follow of the given grammer?**

E → TE′

E′ → *TE′|ε

T → FT′

T′ → ε|+FT′

F → id | (E)

Grammer	FIRST( )	FOLLOW( )
E→TE′	{id, (}	{$,)}
E′→*TE′|ε	{*, ε}	{$,)}
T→FT′	{id, (}	{+,), $}
T′→ε|+FT′	{+, ε}	{$,), *}
F→id|(E)	{id, (}	{+, *,), $}

Follow(T) = {FIRST(E') - ε}
= {{+, ε} - ε}
= {+}

Follow(T') = FOLLOW(T) = {+,), $}

Q. Given Grammer

S → ACD

C → a, b

Find Follow of A and D

Follow(A) = FIRST(C) = {a, b}

Follow(D) = Follow(S) = {$}

Q. Given grammer find FOLLOW() of A ?

S → ABC

A → DEF

B → ε

C → ε

D → ε

F → ε

Follow(A) = FIRST(B)
= FIRST(C)
= Follow(S)

Follow(A) = {$}

Q. Find first() and follow() of given grammer.

	FIRST()	FOLLOW()
S → ABCDE	{a, b, c}	{a, b, c}
A → a | ε	{a, ε}	{b, c}
B → b | ε	{b, ε}	{a}
C → c	{c}	{d, e, $}
D → d | ε	{d, ε}	{e, $}
E → e | ε	{e, ε}	{$}

Q. FIND first() & follow().

	FIRST	FOLLOW
S → BbICd	{a, b, c, d}	{$}
B → aB | ε	{a, ε}	{b}
C → cC | ε	{c, ε}	{d}

Q. Find first() and follow()

	FIRST	FOLLOW
S→ACB|CbBa	{d, g, h, ε, b, a}	{$}
A→da | BC	{d, g, h, ε}	{h, g, $}
B→g|ε	{g, ε}	{$, a, h, g}
C→h|ε	{h, ε}	{g, $, b, h}

Q. Find First() and Follow()

	First()	Follow()
S→aBDh	{a}	{$}
B→cC	{c}	{g, f, h}
C→bC|ε	{b, ε}	{g, f, h}
D→Ef	{g, f, ε}	{h}
E→g|ε	{g, ε}	{f, h}
F→f|ε	{f, ε}	{h}

Q. Find First() & Follow().

	FIRST()	Follow()
S→Aa|Ac	{b}	{$}
A→b	{b}	{a, c}

Q. Construct the predictive passing table for grammer:

E→TE'
E'→+TE'|ε
T→FT'
T'→*FT'|ε
F→(E)|id

Firstly find first() and follow() of given grammer.

Symbols	FIRST	FOLLOW
E	{(, id}	{), $}
E'	{+, ε}	{), $}
T	{(, id}	{+,), $}
T'	{*, ε}	{+,), $}
F	{(, id}	{+, *,), $}

	id	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE			E'→ε	E'→ε
T	T→FT'			T→FT'		
T'		T'→*FT'	T'→ε		T'→ε	T'→ε
F	F→id			F→(E)		

S → AaAb | BbBa

A → ε

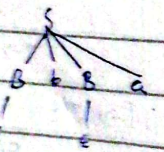
B → ε

Consider the grammer and find first() and follow()

	FIRST	FOLLOW
S	{a, b}	{$}
A	{ε}	{a, b}
B	{ε}	{b, a}

	a	b	$
S	S→AaAb	S→BbBa	
A	A→ε	A→ε	
B	B→ε	B→ε	

For String "ba"



Q. Construct the LL(1) passe.

S → (L) | a

L → L, S | S

⇒ Here left recursion occurs, so first eliminate that:-

Let remove L → S | S

L → SL'

L' → , L | ε

⇒ Now, the grammer is:-

S → (L) | a

L → SL'

L' → , SL' | ε

⇒ Now, find FIRST and FOLLOW

	FIRST	FOLLOW
S	{(, a}	{, , $,)}
L	{(, a}	{)}
L'	{, , ε}	{)}

	a	(	)	,	$
S	S→a	S→(L)			
L	L→SL'	L→SL'			
L'			L'→ε	L'→,SL'	

Q- Construct the passing table for following-

S → A

A → aB | Ad

B → bBC | f

C → g

Here, A → Ad | aB is left recursive.

So, first eliminate them.

S → A

A → aBA'

A' → dA' | ε

B → bBC | f

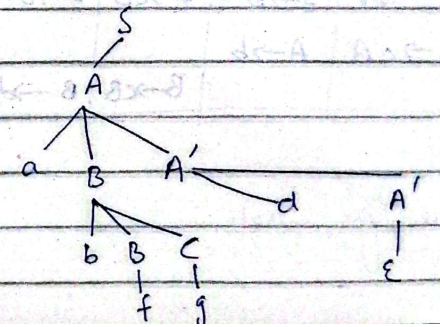
C → g

	FIRST	FOLLOW
S	{a}	{$}
A	{a}	{$}
A'	{d, ε}	{$}
B	{b, f}	{d, g}
C	{g}	{d, g}

The Passing table is-

	a	b	d	f	g	$
S	S→A					
A	A→aBA'					
A'			A'→dA'			A'→ε
B		B→bBC		B→f		
C					C→g	

Consider the string abfgd



Q. Develop predictive pars... for the following gramm...

$S' \rightarrow S$

$S \rightarrow aA \mid b \mid cB \mid d$

$A \rightarrow aA \mid b$

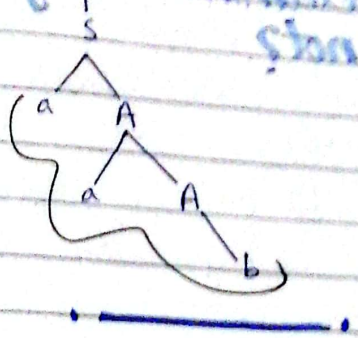
$B \rightarrow cB \mid d$

	FIRST	FOLLOW
S'	{a, b, c, d}	{$}
S	{a, b, c, d}	{$}
A	{a, b}	{$}
B	{c, d}	{$}

The passing table is:-

	a	b	c	d	$
S'	$S' \rightarrow S$	$S' \rightarrow S$	$S' \rightarrow S$	$S' \rightarrow S$	
S	$S \rightarrow aA$	$S \rightarrow b$	$S \rightarrow cB$	$S \rightarrow d$	
A	$A \rightarrow aA$	$A \rightarrow b$			
B			$B \rightarrow cB$	$B \rightarrow d$	

Parse tree for aab



Q. How to determine the grammar is LL(1) or not?

- Every LL(1) grammar is not feasible for passing table.
- if one cell of passing table contains more than one production then the grammar is not LL(1).

Short trick:

- $S \rightarrow \alpha_1 | \alpha_2 | \alpha_3$ (No Null)

  ↓

  $First(\alpha_1) \cap First(\alpha_2) \cap First(\alpha_3) = \phi$ (it is LL(1))

  $\neq \phi$ (Not LL(1))

- $S \rightarrow \alpha_1 | \alpha_2 | \varepsilon$

  ↓ |

  $First(\alpha_1) \cap First(\alpha_2) \cap Follow(\$) = \phi$ (LL(1))

  $\neq \phi$ (Not LL(1))

Q. Determine the grammar is LL(1) or not.

$S \rightarrow aSbS | bSaS | \varepsilon$

	First	Follow
S	{a, b, ε}	{$, b, a}

	a	b	$
S	$S \rightarrow aSbS$	$S \rightarrow bSaS$	$S \rightarrow \varepsilon$
	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$	

→ Passing table

- The given grammar is not LL(1) because LL(1) passes totally confuse which production rule choose from two. Therefore the grammar is not LL(1).

Q. Determine the grammar is LL(1) or not?

$S \rightarrow AaAb | BbBa$

$A \rightarrow \varepsilon$

$B \rightarrow \varepsilon$

	FIRST	FOLLOW
S	{a, b}	{$}
A	{ε}	{a, b}
B	{ε}	{b, a}

Passing Table:

	a	b	$
S	S→AaAb	S→BbBa	
A	A→ε	A→ε	
B	B→ε	B→ε	

So, it is LL(1) grammer.

Q. Check whether the grammer is not LL(1) or not.

S→Aa|ε

A→a

	FIRST	FOLLOW
S	{a}	{$}
A	{a}	{a}

Passing Table:

	a	$
S	S→Aa	
A	A→a	

LL(1) grammer.

Q. Check whether the grammer is LL(1) or not?

S→ aSA|ε

A → c|ε

	FIRST	FOLLOW
S	{a, ε}	{c, $}
A	{c, ε}	{$, c}

Passing table:

	a	c	$
S	S→aSA	S→ε	S→ε
A		A→c, A→ε	A→ε

Two different entries,

So, it is not LL(1).

Q. Is the following grammer suitable for LL(1) passing? if not make it suitable for LL(1) passing. Compute First and Follow set. Generate the passing table.

$S \rightarrow AB$

$A \rightarrow Ca \mid \varepsilon$

$B \rightarrow BaAC \mid c$

$C \rightarrow b \mid \varepsilon$

⇒ The given grammer is not suitable for LL(1) passing. because the production rule

$B \rightarrow BaAC$ is left recursive.

⇒ Lets remove:-

$S \rightarrow AB$

$A \rightarrow Ca \mid \varepsilon$

$B \rightarrow cB'$

$B' \rightarrow aACB' \mid \varepsilon$

$C \rightarrow b \mid \varepsilon$

	FIRST	FOLLOW
S	{b, c, a}	{$}
A	{a, b, ε}	{a, c, b}
B	{c}	{$}
B'	{a, ε}	{$}
C	{b, ε}	{a, $}

The predictive passing table:-

	a	b	c	$
S	$S \rightarrow AB$	$S \rightarrow AB$	$S \rightarrow AB$	
A	$A \rightarrow Ca$ $A \rightarrow \varepsilon$	$A \rightarrow Ca$ $A \rightarrow \varepsilon$	$A \rightarrow \varepsilon$	
B			$B \rightarrow cB'$	
B'	$B' \rightarrow aACB'$			$B' \rightarrow \varepsilon$
C	$C \rightarrow \varepsilon$	$C \rightarrow b$		$C \rightarrow \varepsilon$

So, the given grammer is not LL(1).

Q. Test whether the following grammar is LL(1) or not. Construct the predictive passing table.

$S \rightarrow 1AB|\varepsilon$

$A \rightarrow 1AC|0C$

$B \rightarrow 0S$

$C \rightarrow 1$

	FIRST	FOLLOW
S	{1, ε}	{$, ●}
A	{1, 0}	{1, 0}
B	{0}	{$}
C	{1}	{0, $}

The predictive passing table:-

	0	1	$
S	~~S→1AB~~	S→1AB	S→ε
A	A→0C	A→1AC	
B	B→0S		
C		C→1	

So, the given grammar is LL(1).

Q. Draw the passing table for table driven passes for the given grammar. Is the grammar LL(1)?

$A \rightarrow AaB|x$

$B \rightarrow BCb|Cy$

$C \rightarrow Cc|\wedge$

Let remove the left recursion.

∧→ is equal to ε.

$A \rightarrow xA'$

$A' \rightarrow aBA'|\varepsilon$

$B \rightarrow BCb$

$B \rightarrow Cy$

$C \rightarrow AC'$

$C' \rightarrow cC'|\varepsilon$

	a	b	c	x	y	$
A				A→x		
B'			B→BCb		B→Cy B→BCb	
C			C→Cc C→∧			

	FIRST	FOLLOW
A	{x}	{a, $}
B	{c, y, ε}	{b, c}
C	{c, ∧}	{y, b}

⇒ As there are multiple entries in c and y column, given grammer is not LL(1).

Q. For the following grammer

D → TL;

L → L, id|id

T → int | float

1) Remove left recussion (if required).
2) Find first and follow for each non-terminal for resultant grammer.
3) Construct LL(1) parsing table.
4) Parse the following string and Parse tree for:

int id, id;

1) Formula for removing left recussion:

A → Aα | β is the rule.

A → β A'

A' → αA' | ε

D → TL;	No action as the rule is not left recussive.
L → L, id|id	L → id L'
	L' → , id L'
	L' → ε
T → int|float	No left action as the rule has no recussive.

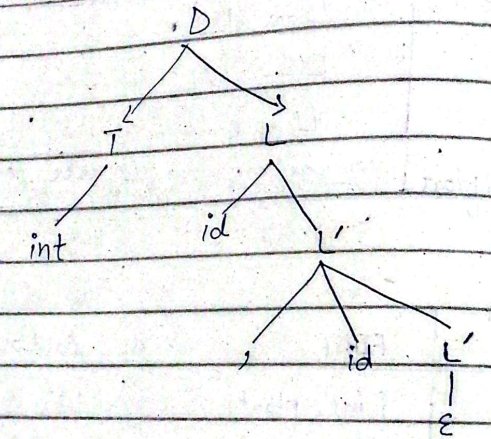
2)

	FIRST	FOLLOW
D	{int, float}	{$}
L	{id}	{;}
L'	{, , ε}	{;}
T	{int, float}	{id}

3) Construction of parsing Table.

	id	,	int	float	;
D			D→TL;	D→TL;	
L	L→idL'				
L'		L'→,idL'			L→ε
T			T→int	T→float	

4) Passing of String int id, id;



Q. Check following grammer is LL(1) or not

$S \rightarrow aB \mid \varepsilon$

$B \rightarrow bC \mid \varepsilon$

$C \rightarrow cS \mid \varepsilon$

Symbol	FIRST	FOLLOW
S	{a, ε}	{$}
B	{b, ε}	{$}
C	{c, ε}	{$}

The passing table will be:

	a	b	c	$
S	S→aB			S→ε
B		B→bC		B→ε
C			C→cS	C→ε

Yes, the given grammer is LL(1).

Q. Write necessary conditions for LL(1) grammers?

- free from left recursion.
- free from ambiguity
- left factored.

Q. Consider the following grammer (PP-2019)

$X \to Y a Y b \mid Z b Z a$

$Y \to \varepsilon$

$Z \to \varepsilon$

Using the definition of LL(1), explain why the grammer is or not LL(1)?

	First	Follow
X	{a, b}	{$}
Y	{ε}	{a, b}
Z	{ε}	{a, b}

Passing table:

	a	b	$
X	$X \to YaYb$	$X \to ZbZa$	
Y	$Y \to \varepsilon$	$Y \to \varepsilon$	
Z	$Z \to \varepsilon$	$Z \to \varepsilon$	

∘ The given grammer is LL(1). because each cell of table contains only one production.

Q. write Short cut Technique for LL(1)?

⇒ A Grammes without ε is LL(1) if for every production of the $A \to \alpha_1 \mid \alpha_2 \mid \alpha_3 \mid \dots \mid \alpha_n$, the set First($\alpha_1$), First($\alpha_2$), First($\alpha_3$), ..., First($\alpha_n$) are mutually disjoint.

∘ $First(\alpha_1) \cap First(\alpha_2) \cap \dots First(\alpha_n) = \phi$

⇒ A Grammes with ε is LL(1) if for every production of the $A \to \alpha \mid \varepsilon$

∘ $First(\alpha) \cap Follow(A) = \phi$